File 12 - Orchestrator Workflow and Approval Rules

AgileFlow AI - Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation

Purpose of this document

This document defines how the Central SDLC Orchestrator controls the AgileFlow AI implementation workflow, approval gates, agent handovers, failure routing, retry loops, Shared Knowledge Base updates, and final completion rules. It must be followed by the implementation foundation team and all component teams.

1. Orchestrator Role in the System

The Central SDLC Orchestrator is the controller of AgileFlow AI. It does not perform requirement extraction, design generation, code generation, or testing by itself. Instead, it coordinates the correct agent at the correct time, controls human approval gates, records state, routes failures, and keeps the Shared Knowledge Base consistent.

- Controls the approved SDLC workflow sequence.
- Triggers each agent only when required inputs are available.
- Moves project state from one stage to the next after human approval.
- Routes failed requirements, failed builds, failed tests, and security issues to the correct component.
- Records workflow events, comments, retries, status changes, and final decisions.
- Preserves traceability between requirements, design, tests, code, bugs, security findings, and reports.

2. Core Orchestrator Principles

ID	Principle	Implementation Meaning
P1	No automatic progression without approval	The system must not move beyond H1-H5 gates until the related human approval is recorded.
P2	One active workflow stage at a time	A project should have one main active stage to avoid inconsistent outputs.
P3	Shared Knowledge Base is the source of truth	Agents must read approved inputs and write outputs through agreed collections and schemas.
P4	Failure must be routed, not hidden	Build failures, test failures, requirement issues, and security findings must become workflow events.
P5	Retry loops are bounded	Automatic retry/self-fix loops must have a limit; unresolved issues require human review.
P6	Traceability is mandatory	Every accepted artifact must connect to relevant requirement, design, code, test, bug, or security IDs.

3. Workflow Stage Order

The workflow must follow the sequence below. The implementation team should use these stage names consistently in the backend, frontend, database, and UI status badges.

No.	Workflow Stage	Meaning	Main Output
01	PROJECT_CREATED	Project is created with basic client idea and metadata.	Project record, client input
02	REQUIREMENTS_IN_PROGRESS	Requirement Gathering and Analysis Agent is processing client input.	Draft requirements
03	H1_WAITING_APPROVAL	Human reviews generated requirements and quality report.	H1 decision
04	DESIGN_IN_PROGRESS	Project Management and Design Agent generates planning/design artifacts.	Backlog/design package
05	H2_WAITING_APPROVAL	Human reviews design, backlog, API, DB, UI, and traceability outputs.	H2 decision
06	TEST_PLANNING_IN_PROGRESS	Testing Agent creates pre-code test specifications and mappings.	Test package
07	H3_WAITING_APPROVAL	Human reviews generated test specifications before implementation.	H3 decision
08	IMPLEMENTATION_IN_PROGRESS	Implementation and Git Automation Agent generates prototype modules and logs.	Code metadata/logs
09	H4_WAITING_APPROVAL	Human reviews generated implementation and execution status.	H4 decision
10	VALIDATION_IN_PROGRESS	Testing Agent executes tests and security validation after implementation.	Validation results
11	H5_WAITING_APPROVAL	Human reviews test report, security report, bug report, and validation evidence.	H5 decision
12	COMPLETED	Final outputs and traceability report are approved.	Final report package

4. Human Approval Gates

Approval gates preserve the human-in-the-loop principle. Each gate must support Approve, Reject, and Request Changes decisions with a reviewer comment.

Gate	Approval Name	Responsible Output Source	Reviewer Checks	If Approved
H1	Requirement Approval	Requirement Gathering and Analysis Agent	SRS, requirements, user stories, acceptance criteria, quality report	Move to DESIGN_IN_PROGRESS
H2	Design and Planning Approval	Project Management and Design Agent	Backlog, sprint plan, UML/API/DB/UI artifacts, traceability	Move to TEST_PLANNING_IN_PROGRESS
H3	Pre-Code Test Approval	Testing, Validation and Security Agent - Pre-Code	Test specs, test skeletons, security test cases, test mapping	Move to IMPLEMENTATION_IN_PROGRESS
H4	Implementation Approval	Implementation and Git Automation Agent	Generated prototype modules, repo/branch	Move to VALIDATION_IN_PROGRES

			metadata, build/runtime logs	S
H5	Final Validation Approval	Testing, Validation and Security Agent - Post-Code	Test report, security report, bug report, final validation evidence	Move to COMPLETED

5. Approval Decision Rules

Decision	Meaning	Required Orchestrator Action
APPROVE	Reviewer accepts the current output.	Store approval record, lock current artifact version, update workflow stage, notify next component.
REJECT	Reviewer decides output is not acceptable.	Mark stage as REJECTED, record reason, keep current stage blocked, route back to responsible agent.
REQUEST_CHANGES	Reviewer accepts concept but asks for corrections.	Create change request event, keep gate pending, allow agent to regenerate or revise affected artifact.
HOLD	Reviewer pauses the workflow.	Set status to ON_HOLD, prevent next-step execution until released by authorized reviewer.

Implementation rule

Every approval decision must include project_id, gate_id, reviewer_name or reviewer_role, decision, comment, timestamp, artifact_version, and next_stage. The system should not silently approve any output.

6. Next-Step Transition Rules

Current Stage	Trigger	Minimum Condition	Next Stage
PROJECT_CREATED	Start requirements	Client input exists	REQUIREMENTS_IN_PROGRESS
REQUIREMENTS_IN_PROGRESS	Requirements generated	Requirement package saved	H1_WAITING_APPROVAL
H1_WAITING_APPROVAL	H1 approved	Approval decision = APPROVE	DESIGN_IN_PROGRESS
DESIGN_IN_PROGRESS	Design package generated	Design artifacts saved	H2_WAITING_APPROVAL
H2_WAITING_APPROVAL	H2 approved	Approval decision = APPROVE	TEST_PLANNING_IN_PROGRESS
TEST_PLANNING_IN_PROGRESS	Test package generated	Test package saved	H3_WAITING_APPROVAL
H3_WAITING_APPROVAL	H3 approved	Approval decision = APPROVE	IMPLEMENTATION_IN_PROGRESS
IMPLEMENTATION_IN_PROGRESS	Prototype generated/run	Code metadata and logs saved	H4_WAITING_APPROVAL
H4_WAITING_APPROVAL	H4 approved	Approval decision = APPROVE	VALIDATION_IN_PROGRESS
VALIDATION_IN_PROGRESS	Validation completed	Reports saved	H5_WAITING_APPROVAL
H5_WAITING_APPROVAL	H5 approved	Approval decision = APPROVE	COMPLETED

7. Failure Routing Rules

The Orchestrator must route issues based on the type of failure. It should create a workflow event and a task/bug/security record before sending the issue to an agent.

Failure Type	Example	Route To	Expected Resolution

Requirement issue	Ambiguous, missing, conflicting, duplicated requirement	Requirement Gathering and Analysis Agent	Clarification request, updated requirement package
Design/planning issue	Wrong backlog, missing API, inconsistent DB/UI mapping	Project Management and Design Agent	Updated design package, Jira/mock issue update
Build/runtime failure	Dependency error, syntax error, config error, simple runtime failure	Implementation and Git Automation Agent	Log analysis, limited self-fix, rerun result
Test failure	Unit/API/integration/UI/regression test failed	Orchestrator -> Design Agent + Implementation Agent	Jira/mock bug update, code fix, retest request
Security issue	Dependency risk, secret leak, weak authorization, input validation issue	Orchestrator -> Design Agent + Implementation Agent	Issue tracking, code/design fix, security re-check
Traceability gap	Requirement lacks linked design/code/test/report artifact	Responsible artifact owner	Updated traceability links

8. Retry Loop Rules

- Each automatic retry loop must store retry_count and last_error_summary.
- Recommended maximum automatic retries: 3 for build/runtime self-fix and 2 for test/security fix cycles.
- If retry limit is exceeded, status must change to HUMAN_REVIEW_REQUIRED.
- Retry attempts must not overwrite previous logs; each attempt should create a new log record.
- The Orchestrator must only retry the affected stage, not restart the whole project workflow.
- After a successful fix, the workflow should return to the validation/review point that failed.

Loop	Allowed Automatic Action	Maximum Retry	Escalation
Requirement clarification loop	Regenerate after human clarification	Until approved manually	Supervisor/client review
Build/runtime self-fix loop	Limited syntax, dependency, config, simple runtime fixes	3	Implementation review
Test failure correction loop	Route bug to Design/Implementation then retest	2	Human technical review
Security correction loop	Route security issue to Design/Implementation then rescan	2	Security reviewer decision

9. Shared Knowledge Base Update Rules

All components must use the Shared Knowledge Base as the single source of truth. The Orchestrator should verify that each stage stores its output before moving forward.

Collection	Stores	Update Moment
projects	Project metadata, current stage, overall status	Project creation and every stage transition
requirements	Approved and draft requirements with REQ IDs	After Requirement Agent generation and H1 approval
design_artifacts	Backlog, sprint plan, UML/API/DB/UI/wireframes	After Design Agent generation and H2 approval
test_cases	Pre-code test specs, executable skeletons, test IDs	After Test Planning generation and H3 approval
implementation_artifacts	Code module metadata, repo info, branch/commit data	After Implementation Agent generation and H4 approval
execution_logs	Build logs, run logs, agent execution logs	During implementation, validation, and retries
test_results	Executed test results,	After post-code validation

	pass/fail/skipped/coverage	
security_findings	Security scan results and vulnerability records	After security validation
bug_reports	Bug reports generated from failed tests or reviews	When failures are detected
approvals	H1-H5 decisions and comments	Every approval gate decision
traceability_links	Requirement-design-test-code-bug-security-report links	Every artifact creation and final report generation
workflow_events	Stage changes, failures, approvals, retries, handovers	Every Orchestrator action

10. Workflow Event Logging Rules

Every important Orchestrator action must create a workflow event. This creates evidence for debugging, evaluation, and final research documentation.

- Create an event when a project is created.
- Create an event when an agent starts and completes work.
- Create an event when an artifact is generated, revised, approved, rejected, or locked.
- Create an event when a failure is detected and routed.
- Create an event for each retry attempt and final retry result.
- Create an event when final reports are generated and H5 is approved.

Field	Meaning
event_id	Unique workflow event ID, e.g., EVT-001
project_id	Project linked to the event
event_type	AGENT_STARTED, ARTIFACT_GENERATED, APPROVAL_DECISION, FAILURE_ROUTED, RETRY_ATTEMPT, STAGE_CHANGED
source_component	Component that produced the event
target_component	Component receiving the next action, if any
stage_before / stage_after	Workflow transition evidence
message	Short human-readable explanation
metadata	IDs, retry counts, error summary, artifact version, report links
created_at	Timestamp

11. Orchestrator API Rules

The backend should expose Orchestrator APIs that support workflow state, next-step execution, approval decisions, failure routing, and traceability retrieval.

Method	Endpoint	Purpose
GET	/api/projects/{projectId}/workflow	Return current stage, approvals, active agent, retry status, and recent events.
POST	/api/projects/{projectId}/orchestrator/next-step	Validate current stage and move the project to the next allowed action.
POST	/api/projects/{projectId}/approvals/{gateId}	Record H1-H5 approval decision and update workflow state.
POST	/api/projects/{projectId}/orchestrator/route-failure	Create failure event and route issue to responsible component.
POST	/api/projects/{projectId}/orchestrator/retry	Create retry attempt for a failed stage if retry limit allows it.

GET	/api/projects/{projectId}/traceability	Return requirement-to-design-to-test-to-code-to-report links.
GET	/api/projects/{projectId}/workflow-events	Return workflow audit trail for UI and reports.

12. Frontend UI Behavior Rules

- Only show the action button that is valid for the current workflow stage.
- Show locked/approved stages as read-only unless a reviewer requests changes.
- Every approval gate screen must show artifact summary, reviewer comment box, and Approve/Reject/Request Changes buttons.
- The Orchestrator dashboard must show current stage, active agent, pending approval, failures, retry count, and recent events.
- Use consistent status badges: PENDING, IN_PROGRESS, WAITING_FOR_APPROVAL, APPROVED, REJECTED, FAILED, COMPLETED, HUMAN_REVIEW_REQUIRED.
- If an API returns a workflow rule violation, show a clear message instead of allowing invalid progression.

13. Implementation Pseudocode

```
function moveToNextStep(projectId):
    project = loadProject(projectId)
    stage = project.current_stage

    if stage requires approval:
        if approval not approved:
            return error("Human approval is required before continuing")

    nextStage = resolveNextStage(stage)
    validateRequiredArtifacts(projectId, stage)
    updateProjectStage(projectId, nextStage)
    createWorkflowEvent(projectId, "STAGE_CHANGED", stage, nextStage)
    notifyResponsibleComponent(projectId, nextStage)
    return workflowState(projectId)

function routeFailure(projectId, failureType, source, details):
    target = resolveTargetComponent(failureType)
    createFailureRecord(projectId, failureType, source, target, details)
    createWorkflowEvent(projectId, "FAILURE_ROUTED", source, target)
    updateProjectStatus(projectId, "FAILED" or "HUMAN_REVIEW_REQUIRED")
    return assignedResolutionTask(target)
```

14. Development Checklist

Area	Checklist Item
Workflow stages	All stage names implemented exactly as documented.
Approval gates	H1-H5 approval APIs, UI buttons, reviewer comments, and approval records implemented.
Failure routing	Requirement, design, build/runtime, test, security, and traceability failures routed correctly.
Retry logic	Retry counters and escalation rules implemented.
Shared Knowledge Base	All agents read/write through agreed collections.
Workflow events	Every stage transition, approval, failure, retry, and final completion recorded.
Traceability	Final report can show Requirement -> Design -> Test -> Code -> Bug/Security -> Status.
UI restrictions	Invalid next steps are disabled or blocked with clear messages.

AI agent safety	No AI coding agent should bypass these workflow rules or modify approval logic without review.

15. Final Rule

Final rule for implementation

The Orchestrator must never allow the project to skip human approval, overwrite approved artifacts without a new version, or complete the workflow without final validation evidence. AgileFlow AI is valuable because automation and human control work together.